

RAJALAKSHMI ENGINEERING COLLEGE

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

AI23231 – PRINCIPLES OF ARTIFICIAL INTELLIGENCE LAB

EXPERIMENT 1b

IMPLEMENTATION OF DEPTH FIRST SEARCH

Aim:

To implement depth first search.

Procedure:

1. Start the program.
2. Define a Graph class to represent the graph using an adjacency list.
3. Initialize the graph with the required number of vertices.
4. Add edges between the vertices using the add_edge() function.
5. Create a visited list to keep track of visited vertices.
6. Start the DFS traversal from the specified vertex.
7. Mark the current vertex as visited and display it.
8. Recursively visit each unvisited adjacent vertex.
9. Continue the process until all reachable vertices have been visited.
10. Stop the program.

Program:

```
# DFS Algorithm Implementation

class Graph:
    def __init__(self, vertices):
        self.vertices = vertices
        self.graph = {i: [] for i in range(vertices)}

    # Function to add an edge to the graph
    def add_edge(self, u, v):
        self.graph[u].append(v)
        self.graph[v].append(u)

    # Recursive function for DFS traversal
    def dfs(self, v, visited):
        print(v, end=" ")
        visited[v] = True

        # Recur for all the adjacent vertices
        for neighbor in self.graph[v]:
            if not visited[neighbor]:
                self.dfs(neighbor, visited)

# Main function to demonstrate DFS
def depth_first_search():
    # Creating a graph with 6 vertices
    g = Graph(6)
    # Adding edges
```

```
g.add_edge(0, 1)
g.add_edge(0, 2)
g.add_edge(1, 3)
g.add_edge(1, 4)
g.add_edge(2, 5)

# To track visited nodes
visited = [False] * g.vertices

# Start DFS traversal from vertex 0
print("Depth First Search starting from vertex 0:")
g.dfs(0, visited)

# Calling the main function
depth_first_search()
```

Output:

```
Depth First Search starting from vertex 0:
0 1 3 4 2 5
```

Result: Thus, the Depth First Search algorithm was implemented and executed successfully.